ELF Linker Loader Primer

ELF、`SysV ABI`、静的リンク、動的リンク、`_start` から `main` までの起動経路を、Linux x86-64 上の最小例でつなぐ学習用ノートです。`compilerbook` の次に読むことを想定し、コンパイラが吐いた `.o` が、どのように実行可能ファイルとプロセスへ変わるのかを 1 冊で追えるように構成しています。

> **編集注記**
> この文書は仕様書や source tree の逐語訳ではありません。`ELF gABI`、`AMD64 psABI`、GNU toolchain、glibc、Linux の公開資料をもとに、学習の主線が見える順に再構成した版です。完全な網羅よりも、「誰が、いつ、どこを書き換えるのか」を説明しやすい順序を優先しています。

目次

Contents

はじめに 2
- この教材の主題 3
- なぜ今リンクとローダなのか 3
- 対象と範囲 3
- 2026 年 3 月 10 日時点の toolchain snapshot 4
- 参照する一次資料 4
- 各章の見取り図 4

ELF とオブジェクトファイル 5
- まず観察対象を固定する 5
- `.o` は未完成な部品 5
- section が表すもの 6
- ELF header と section header 6
- symbol table は何を持つか 6
- relocation entry は「あとで埋める印」 6
- `objdump` と `readelf` の役割分担 7
- section header と program header は別物 7
- この章で押さえるべき最小像 7

ABI と静的リンク 7
- ABI は「外の世界との約束」 8
- SysV AMD64 calling convention の最低限 8
- 未解決シンボルはどこで解決されるか 8
- 静的リンクの仕事 8
- 静的ライブラリ `.a` とは何か 9
- よく出る relocation の見え方 9
- 最小リンカの視点 9
- `compilerbook` との接続 9
- この章の出口 10

共有ライブラリと動的リンク 10

なぜ共有ライブラリが必要か 10
PIC は何を守っているか 10
GOT はアドレスの表 10
PLT は関数呼び出しの踏み台 11
遅延束縛とは何か 11
dynamic loader の責務 11
DT_NEEDED と dependency graph 11
静的リンクとの違いを一文で言うと 11
この章の出口 12
ローダとプロセス起動 12
execve は何を始めるか 12
program header は loader の地図 12
PT_INTERP が意味するもの 13
初期 stack に何が積まれるか 13
_start は誰のコードか 13
__libc_start_main の役割 13
main から先も startup の一部である 13
kernel loader と dynamic loader の境界 14
この章の出口 14
付録と次の一歩 14
章ごとの復習ポイント 14
観察コマンドの使い分け 15
小さな演習課題 15
よくあるつまずき 15
.text を読めば実行時の全体が分かると思ってしまう 15
GOT と symbol table を混同する 15
PLT を単なる無駄なジャンプだと思ってしまう 16
section と segment を混同する 16
main が最初の関数だと思ってしまう 16
用語小事典 16
次に読むなら 16
参考資料 16
おわりに 17

はじめに

この文書は、コンパイラが出力した .o がどのように実行可能ファイルへ結び付けられ、execve 後に最初のユーザ空間コードがどのように走り始めるのかを整理するためのノートです。ソースコードをアセンブリへ落とすところまでは理解できても、その先にある ELF、再配置、共有ライブラリ、ローダ、crt1.o、__libc_start_main は別の話に見えがちです。本書の役割は、その切れ目を埋めることです。

低レイヤ学習では、コンパイラ、OS、runtime、観測ツールが別々の話題に見えやすいですが、実際には強くつながっています。compilerbook はソースから機械語へ至る道を見せてくれます。rust-os-book はそのコードが動く土台を扱います。sanitizer-fuzzer-book と go-runtime-primer は、生成されたコードや runtime がどのように観測されるかを扱います。その間をつなぐのが、実行形式とリンクの層です。

この教材の主題

本書の主線は次の 4 点です。

1. `.o` の中に何が入っているか
2. リンカが何を解決し、何を書き換えるか
3. 共有ライブラリと `PLT/GOT` が何をしているか
4. `execve` 後に `_start` から `main` へどう到達するか

この 4 点がつながると、`nm` や `readelf` の出力が単なる表ではなくなります。未解決シンボル、再配置エントリ、`PT_INTERP`、`DT_NEEDED`、`_start` といった名前が、一つのプロセス起動の物語として読めるようになります。

この章の到達点

本書を読み終えるころには、少なくとも次を説明できる状態を目指します。

- `ELF header`、section、program header の役割の違い
- `.o` が未完成な部品であり、再配置がその印であること
- 静的リンクと動的リンクの違いを「誰がいつ解決するか」で説明できること
- `PLT` と `GOT` が間接参照と遅延束縛のために使われること
- `execve`、dynamic loader、`crt1.o`、`__libc_start_main`、`main` のつながり

なぜ今リンクとローダなのか

`compilerbook` の段階では、コンパイラが正しいアセンブリを出力し、ツールチェインが最終的に実行形式へまとめてくれることを前提に進みます。その前提自体は正しいのですが、リンクとローダの層が見えていないと、いくつかの重要な疑問が残ります。

- なぜ外部関数呼び出しが成立するのか
- なぜ同じ `printf` 呼び出しでも静的リンクと動的リンクで見え方が違うのか
- なぜ `main` ではなく `_start` が実際の入口なのか
- なぜ `.text` を眺めるだけでは起動時の全体像が見えないのか

これらは別々の細部ではありません。コンパイラが出した記号付きの部品を、リンカとローダが段階的に完成形へ寄せていくという一つの話です。本書では、その段階性を崩さないように進めます。

対象と範囲

対象は Linux x86-64 上の GNU toolchain に固定します。`ELF` は他の環境でも使われますが、初学段階では一般論を厚くし過ぎると、肝心の「この 1 つの環境で実際に何が起きているか」がぼやけます。そこで本書では、`System V AMD64 ABI`、glibc、`ld-linux-x86-64.so.2` を前提に、観察可能な具体例だけで話を進めます。

一方で、次は本書の主題にはしません。

- `PE` や `Mach-O` との比較の詳細
- linker script の高度な活用
- `LTO` や `ThinLTO`
- kernel 内部の `execve` 実装の完全読解
- `TLS` や `IFUNC` や `symbol versioning` の細部

これらは重要ですが、初版の目標は「実行形式の最小地図を持つこと」です。細部の拡張より、主線の一貫性を優先します。

2026 年 3 月 10 日時点の toolchain snapshot

本書の説明は 2026 年 3 月 10 日時点の Linux x86-64 環境を基準にしています。本文で前提にする観察系ツールは、手元では次の版でした。

- `gcc 13.3.0`
- `GNU ld 2.42`
- `GNU readelf 2.42`
- `glibc 2.39`

出力形式の細部や最適化の影響は版ごとに少し変わります。しかし、本書が重視するのは関数名や行番号の一致ではなく、責務の境界です。したがって、違う版で読んでも「section は部品の都合で、program header は loader の都合」「`PLT/GOT` は間接参照と遅延束縛のため」といった骨格が変わらないように書きます。

> **注意**
> `readelf` や `objdump` の表示順やラベルは版差分の影響を受けます。本文の断片出力は「この行を暗記する」ためではなく、「どの欄に何が出るか」を見失わないための道標として読んでください。

参照する一次資料

本書では、次の一次資料を主参照にします。

- `ELF gABI`
- `System V AMD64 psABI`
- `man 5 elf`
- `man 2 execve`
- `ld.so(8)`
- GNU binutils の `ld`, `readelf`, `objdump` 関連文書
- glibc の `csu` と startup 周辺 source

これらをそのまま並べると、仕様、ユーザ空間 runtime、ツールチェイン、OS の資料が分散して見えます。しかし実際には、同じ起動経路の別の断面を説明しているに過ぎません。本書では、その断面を「部品」「結合」「遅延解決」「起動」に並べ直します。

> **編集注記**
> 最初から `ld.so` や glibc startup source を深追いすると、関心が細部へ散りやすくなります。本書ではまず縮小した観察対象を決め、最小の `.o`、最小の実行ファイル、最小の共有ライブラリという順で世界を広げます。

各章の見取り図

本書は次の順で進みます。

1. `.o` を観察し、section、symbol、relocation の意味を押さえる
2. `SysV ABI` と静的リンクを通して、リンカが何を埋めるのかを見る
3. 共有ライブラリ、`GOT`、`PLT`、遅延束縛を理解する
4. `execve`、program header、dynamic loader、`_start`、`main` をつなぐ

5. 付録で観察コマンド、典型的なつまずき、次に読む source を整理する

この順序には理由があります。program header と loader の話は魅力的ですが、その前に「なぜ未解決の参照が残るのか」が見えていないと、動的リンクの必要性が腹落ちしません。そこで本書では、まず .o の未完成さから始めます。

ELF とオブジェクトファイル

リンクとローダの話を始めるには、まず観察対象を小さく固定する必要があります。本章では、単一の C ファイルと、外部関数呼び出しを含む複数ファイルの最小例を頭に置きながら、.o の中に何が入っているかを整理します。主題は「完成した実行ファイルを読むこと」ではなく、「まだ未完成な部品に、どのような情報が残されているか」を見ることです。

この章の到達点

この章の到達点は次の通りです。

- .o が機械語だけではなく symbol と relocation を持つと説明できる
- section と program header の違いを説明できる
- `.text`、`.data`、`.bss`、`.rodata` の役割を言い分けられる
- `readelf` と `objdump` をどう使い分けるか分かる

まず観察対象を固定する

本書で繰り返し使う最小例は次の 2 つです。

```
// single.c
int main(void) {
  return 42;
}
```

```
// main.c
int ext(void);

int main(void) {
  return ext() + 1;
}
```

```
// ext.c
int ext(void) {
  return 41;
}
```

最初の例は、section の基本を見るのに向いています。二つ目の例は、未解決シンボルと relocation を見るのに向いています。以後の章でも、この小ささをなるべく保ちます。大きなプログラムをいきなり読まないのは、仕組みを減らして役割を見やすくするためです。

.o は未完成な部品

コンパイラやアセンブラが吐く .o は、まだ完成した実行形式ではありません。そこには命令列や静的データが入っていますが、同時に「この場所はあとで直してほしい」「この名前は外で定義される」という情報も残っています。つまり .o は、実行可能というより **結合待ちの部品** です。

この観点を持つと、.o の見え方が変わります。`.text` だけを見て「命令はもう全部ある」と思っても、外部関数呼び出しやグローバル参照の値はまだ確定していないかもしれません。未確定な場所があるからこそ、symbol table と relocation table が必要になります。

section が表すもの

`.o` の内部は、多くの場合 section という単位で整理されています。初学者がまず覚えるべき section は次です。

- `.text` 命令列
- `.rodata` 読み取り専用データ。文字列リテラルや定数表など
- `.data` 初期値付きの書き換え可能データ
- `.bss` 初期値 0 の大域データ。ファイル内にはサイズ情報だけを持つことが多い
- `.symtab` symbol table
- `.rela.text`, `.rela.data` relocation entry

section は、主に **部品を組み立てる側** の都合で分かれています。命令、定数、未解決参照、symbol 情報を別々に持っておくと、リンカが扱いやすいからです。

ELF header と section header

`ELF` 形式のファイル先頭には `ELF header` があります。ここには「これは ELF である」「32bit か 64bit か」「実行形式か共有オブジェクトか relocatable object か」「section header はどこか」といった、全体を読むための入口情報が入っています。

一方、section header table は「各 section がファイル内のどこにあり、どのくらいの大きさで、どんな属性か」を列挙します。`.o` を読む段階では、section header がとても重要です。なぜなら、この段階の主役は loader ではなく linker だからです。linker は section を単位に部品を集め、symbol を解決し、relocation を適用します。

symbol table は何を持つか

symbol table は、名前とその意味付けの表です。ただし、ここでいう「意味」は単にアドレスではありません。代表的には次の情報を持ちます。

- 名前
- どの section に属するか
- 値またはオフセット
- 大域か局所か
- 関数かオブジェクトか
- 未定義か定義済みか

たとえば `ext` は `main.o` 側では未定義 symbol として現れますが、`ext.o` 側では `.text` 内の定義済み symbol として現れます。linker の仕事は、これらを照合して「`main.o` のこの参照は `ext.o` のこの定義へつながる」と決めることです。

relocation entry は「あとで埋める印」

relocation entry は、ファイルのある位置に入っている仮の値を、後で正しい値へ直すための指示です。外部関数呼び出し、グローバル変数参照、アドレス定数などで現れます。重要なのは、relocation は「値そのもの」ではなく **値の作り方** を表している点です。

たとえば x86-64 では、次のような違いがあります。

- 絶対アドレスを書き込みたいのか
- 現在位置からの相対オフセットを書き込みたいのか
- `PLT` を経由する呼び出しとして扱いたいのか

この違いが relocation type に現れます。つまり relocation table を見ると、linker や dynamic loader が後でどういう計算をするつもりなのかが見えてきます。

`objdump` と `readelf` の役割分担

最初に使い分けたい道具は `readelf` と `objdump` です。

- `readelf` ELF header、section header、symbol、relocation のような構造を見る
- `objdump` 逆アセンブルして、命令列と参照の位置関係を見る

たとえば `call` 命令が `ext` を呼んでいると聞いても、`objdump -dr` で命令位置を見なければ「どこに relocation が刺さるのか」が分かりにくいです。逆に `objdump` だけ見ても、symbol が未定義なのか、どんな relocation type が付いているのかは見落としやすいです。両方を往復するのが基本になります。

section header と program header は別物

ここで非常に重要なのが、section header と program header を混同しないことです。section は linker が部品を扱うための単位でした。これに対して program header は、実行時に loader が何をどこへ写像するかを表します。

この違いは、`.o` と実行形式の役割の違いそのものです。

- `.o` では section が主役
- 実行形式や shared object では program header が主役

section は「部品の都合」、program header は「起動時の写像の都合」です。後者は次章以降で改めて扱いますが、この区別をここで持っておくと、`readelf -S` と `readelf -l` の出力を混同しにくくなります。

> **注意**
> section があるからといって、kernel が section ごとにメモリへ載せるわけではありません。実際に `execve` 後の写像を決めるのは program header です。section は主に static link のためにあり、起動時の直接の単位ではありません。

この章で押さえるべき最小像

本章の要点を縮めると、`.o` は次の 3 層から見れば十分です。

1. 命令や静的データそのもの
2. 名前を付ける symbol table
3. 後で値を直す relocation table

この 3 つが見えると、linker は「不完全な部品を集め、symbol を照合し、必要な位置を書き換える仕事」だと自然に見えてきます。次章では、その書き換えが `ABI` とどう結び付くかを見ます。

ABI と静的リンク

`.o` の中に symbol と relocation があると分かったら、次に必要なのは「どんな値を埋めるのが正しいのか」という約束です。その約束が `ABI` です。本章では `System V AMD64 ABI` を足場に、静的リンクが何を解決しているのかを整理します。

この章の到達点
この章の到達点は次の通りです。

- `ABI` がソース言語ではなく部品同士の約束だと説明できる
- `SysV AMD64` の最低限の calling convention を説明できる
- linker が symbol 解決、section 配置、relocation 適用を行うと理解する
- 静的ライブラリ `.a` が「オブジェクトファイルの束」だと説明できる

ABI は「外の世界との約束」

言語仕様だけでは、関数の引数をどのレジスタへ載せるか、戻り値をどこへ返すか、stack をどうそろえるかは決まりません。C コンパイラ、アセンブラ、リンカ、標準ライブラリ、デバッガが互いに連携できるのは、これらを `ABI` が決めているからです。

重要なのは、`ABI` はコンパイラ内部の都合ではないということです。たとえば `compilerbook` で自作コンパイラを書くときも、外部の `printf` を呼びたいなら `ABI` の約束を守る必要があります。逆に言えば、リンクできるかどうかは「同じ言語で書いたか」より「同じ ABI を話しているか」に強く依存します。

SysV AMD64 calling convention の最低限

本書で必要な最低限は次です。

- 整数系の最初の引数は `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`
- 戻り値は主に `rax`
- 呼び出し前後で保存責務の異なるレジスタがある
- `call` 時点で stack alignment の約束がある

この約束があるから、ある `.o` で生成した関数と、別の `.o` で生成した関数が問題なくつながります。linker は名前を解決しますが、呼び出し自体の意味を成立させているのは ABI です。symbol 解決だけでは、レジスタの使い方までは保証されません。

未解決シンボルはどこで解決されるか

`main.o` の中で `ext` が未定義 symbol として現れるとき、コンパイラやアセンブラの仕事はまだ終わっていません。ただし、この段階で必要な情報はすでにあります。

- `ext` という名前で外部参照したい
- 参照位置は `.text` のこの命令付近である
- 書き換え方はこの relocation type に従う

linker は、入力として渡された複数の `.o` や `.a` を走査し、`ext` の定義を探します。見つかったら、最終配置に基づいて relocation を解決します。見つからなければ undefined reference で失敗します。

静的リンクの仕事

静的リンクを最小化すると、仕事は次の 4 つに分けられます。

1. 入力オブジェクトとライブラリを集める
2. symbol 定義と参照を照合する
3. section を最終レイアウトへ並べる
4. relocation を適用して、未確定な値を書き換える

この 4 つをまとめて見ないと、link は単に「ファイルをくっつける作業」に見えてしまいます。実際には、symbol graph を解き、最終配置に応じてアドレス計算をやり直す処理です。

静的ライブラリ `.a` とは何か

静的ライブラリは、ざっくり言えばオブジェクトファイルの束です。`.a` 自体が特別な命令形式を持つわけではありません。中身は `.o` をまとめた archive で、linker は必要な symbol を満たすメンバだけを取り込みます。

ここで理解しておくと得なのが、「静的リンクは最初から全部展開する」のではなく、必要な member を選びながら進むことです。だからこそ library の指定順が効く場面があります。ある library が別の library の symbol を必要とするなら、解決順を意識しなければなりません。

よく出る relocation の見え方

x86-64 では、初学段階で次の名前を見かけることが多いです。

- `R_X86_64_PC32` 現在位置からの相対値で参照する
- `R_X86_64_PLT32` `PLT` 経由の呼び出しとして扱う
- `R_X86_64_64` 64bit の絶対値を埋める

ここで暗記したいのは番号ではなく、「同じ symbol 参照でも埋め方は一種類ではない」ということです。`call ext` なのか、データ中へアドレスを書き込むのか、`PLT` へ飛ばすのかで、最終的に書き込む値が変わります。relocation type はその区別を表しています。

最小リンカの視点

学習用に静的リンクを縮小して考えるなら、最小リンカは次のように動くとみなせます。

1. 入力 `.o` を全部読み、section と symbol を表へ積む
2. `.text` や `.data` の最終開始位置を決める
3. 定義済み symbol の最終アドレスを計算する
4. relocation entry を順に見て、対応する値を書き込む

もちろん実際の linker は、弱い symbol、共通シンボル、リンクスクリプト、セクション属性、デバッグ情報など、もっと多くを扱います。しかし最初に必要なのは、「symbol を解決して終わり」ではなく、最後に relocation を適用して初めて命令列が完成するという感覚です。

`compilerbook` との接続

`compilerbook` で自作コンパイラを書くと、外部関数呼び出しを含むアセンブリを吐けるようになった段階で linker の存在感が急に増します。自作コンパイラは、`printf` の実装を持っていなくても構いません。必要なのは、正しい symbol 名と正しい呼び出し規約を守った参照を出すことです。残りは linker が受け持ちます。

この視点を持つと、コンパイラの責務と linker の責務がきれいに分かれます。

- コンパイラは「どの symbol をどう参照したいか」を表す
- linker は「その参照が最終的にどこを指すか」を確定させる

> **注意**
> 静的リンクを理解し始めた段階では、「アドレスが決まってから relocation を直す」の順

を意識してください。symbol 名だけ解決しても、section の最終配置が決まっていなければ書き込む値は確定しません。

この章の出口

本章の出口は、静的リンクを「未解決 symbol を持つ部品を、ABI の約束を保ったまま 1 本の実行形式へ閉じる処理」と見られることです。ここまで来ると、次の疑問が自然に現れます。巨大な libc を毎回丸ごと取り込まず、実行時まで解決を遅らせるにはどうするのか。それが次章の動的リンクです。

共有ライブラリと動的リンク

静的リンクが「ビルド時に全部を閉じる」方法だとすると、動的リンクは「一部の解決を実行時まで遅らせる」方法です。本章では共有ライブラリ、`PIC`、`GOT`、`PLT`、遅延束縛を通して、動的リンクで誰が何をしているのかを整理します。

この章の到達点

この章の到達点は次の通りです。

- shared object が再利用可能な部品である理由を説明できる
- `PIC` が必要になる理由を説明できる
- `GOT` と `PLT` の役割の違いを説明できる
- 動的リンカが `DT_NEEDED` や relocation を見て依存関係を解決すると理解する

なぜ共有ライブラリが必要か

静的リンクだけでもプログラムは動きます。しかし、巨大な標準ライブラリを各 executable へ毎回取り込むと、ディスク使用量も更新のコストも増えます。そこで、実行時に共通部品として読み込める shared object が使われます。

このとき重要なのは、shared object は単に「別ファイルに分けた `.text`」ではないことです。実行時にどのアドレスへ載るかは固定されませんし、複数プロセスで共有されることもあります。だからこそ、位置に依存しにくいコードと、実行時の relocation を支える仕組みが必要になります。

`PIC` は何を守っているか

`PIC` は position-independent code の略です。共有ライブラリは、毎回同じ仮想アドレスへ載るとは限りません。したがって、コード中へ絶対アドレスを直接焼き込む形では扱いにくくなります。`PIC` は、相対参照や indirection を多用して、ロード位置が変わっても動くようにする方針です。

ここで大事なのは、`PIC` は「絶対アドレスを一切使わない魔法」ではないということです。必要なアドレス参照は残ります。ただし、それをコード中へ直接散らさず、`GOT` のような表へ集めることで、実行時の修正箇所を限定します。

`GOT` はアドレスの表

`GOT` は Global Offset Table の略です。名前から global variable 専用に見えますが、実際には「実行時に確定するアドレスを間接参照するための表」と考えるのが分かりやすいです。shared object や executable のコードは、必要な先を直接知らず、まず GOT のスロットを参照します。

この 1 段の indirection にはコストがありますが、得られるものも大きいです。ロード位置が変わっても、dynamic loader が修正すべき場所を GOT へ集中させやすくなります。つまり `PIC` と `GOT` は別の概念ですが、実用上は強く結び付いています。

`PLT` は関数呼び出しの踏み台

`PLT` は Procedure Linkage Table の略です。関数呼び出しを shared object 越しに行うとき、call site から直接最終アドレスへ飛ぶのではなく、いったん `PLT` の stub を経由する形が使われます。初回呼び出しではこの stub が dynamic loader へ制御を渡し、symbol 解決後は対応する GOT エントリが更新され、2 回目以降は直接近い形で飛べるようになります。

ここで混同しやすいのは、`PLT` と `GOT` の役割です。

- `GOT` 実行時に確定するアドレスを置く表
- `PLT` 関数呼び出しを一度受けて、必要なら解決処理へ渡す stub 群

関数呼び出しでは両者が協調しますが、同じものではありません。

遅延束縛とは何か

動的リンクでは、全 symbol を起動直後に解決する方法と、必要になるまで遅らせる方法があります。後者が lazy binding です。`PLT` の初回呼び出し時に dynamic loader が symbol を探し、対応する GOT スロットを書き換えることで、その後の呼び出しを速くします。

遅延束縛が腹落ちすると、`PLT` stub の奇妙な見た目にも意味が出てきます。あれは単なる余計なジャンプではなく、「まだ解決されていないなら resolver を呼び、解決済みならそのまま本体へ飛ぶ」ための分岐点です。

dynamic loader の責務

Linux x86-64 では、典型的には `ld-linux-x86-64.so.2` が dynamic loader です。`execve` 後、kernel は `PT_INTERP` が示す interpreter を起動経路へ組み込みます。dynamic loader は主に次の仕事をします。

1. 必要な shared object を読み込む
2. 各 object の依存関係を解決する
3. dynamic relocation を適用する
4. 必要なら `PLT/GOT` を初期化する
5. 最終的な entry point へ制御を渡す

静的リンクでは build 時にやっていた仕事の一部を、ここで実行時に行っているわけです。

`DT_NEEDED` と dependency graph

共有ライブラリ依存は、`ELF` の dynamic section に並ぶ `DT_NEEDED` から見えてきます。これは「この executable または shared object は、実行時にこの shared object を必要とする」という宣言です。dynamic loader はこれをもとに dependency graph をたどり、必要な object をロードしていきます。

重要なのは、linker が build 時に dependency 情報を埋め込み、loader が実行時にそれを読むという分業です。動的リンクは「全部実行時の魔法」ではなく、build 時と実行時の共同作業です。

静的リンクとの違いを一文で言うと

ここまでの違いを一文で縮めると、次のようになります。

- 静的リンク: build 時に symbol 解決と relocation をほぼ閉じる
- 動的リンク: 一部の symbol 解決と relocation を実行時まで持ち越す

この「いつ解決するか」の違いが、shared object、`PIC`、`GOT`、`PLT`、dynamic loader を必要にしています。

> **注意**
> `GOT` は symbol table ではありません。symbol 名を見て検索する表ではなく、解決済みアドレスを置くための表です。名前解決そのものは dynamic loader 側の責務であり、`GOT` はその結果を高速に再利用するための入れ物です。

この章の出口

本章の出口は、`PLT/GOT` を「ややこしい x86-64 の小細工」ではなく、「実行時解決を支える分業」として見られることです。ここまで来ると、次は自然に「では kernel と dynamic loader は、どうやって最初の 1 命令目まで運ぶのか」という疑問になります。次章では `execve` から `_start` までを追います。

ローダとプロセス起動

静的リンクと動的リンクの役割が見えたら、最後にそれらが実行開始とどうつながるかを見ます。本章では `execve`、program header、interpreter、dynamic loader、`crt1.o`、`__libc_start_main` を一本の流れとして整理します。目標は、`main` が「最初に走る関数」ではないことを、手順として説明できるようになることです。

> **この章の到達点**
> この章の到達点は次の通りです。
>
> - `execve` が新しいプロセス像を作る入口だと説明できる
> - program header が loader のための写像情報だと説明できる
> - `PT_INTERP` と dynamic loader の役割を説明できる
> - `_start`、`__libc_start_main`、`main` の呼び出し順を説明できる

execve は何を始めるか

ユーザ空間プログラムの起動は、典型的には `fork` と `execve` の組み合わせで見えますが、実際に新しい実行イメージを作るのは `execve` です。kernel は指定された ELF executable を読み、必要なメモリ領域を用意し、初期 stack を構成し、entry point へ飛べる状態を作ります。

ここで重要なのは、`execve` が単にファイルを読み込むだけではないことです。アドレス空間、初期 stack、補助ベクタ、必要なら interpreter の起動経路まで含めて、新しいプロセス像を作ります。

program header は loader の地図

section header は linker の都合でした。起動段階で主役になるのは program header です。kernel や dynamic loader は、`PT_LOAD`、`PT_DYNAMIC`、`PT_INTERP` などの segment 情報を見て、どの範囲をどんな権限でメモリへ写像するかを決めます。

ここで section と segment の違いが改めて効きます。`readelf -S` で見える section は細かい部品の境界ですが、`readelf -l` で見える segment は起動時の写像単位です。複数の section が 1 つの loadable segment にまとめられることも普通です。

`PT_INTERP` が意味するもの

動的リンク executable には、どの dynamic loader を使うかを示す `PT_INTERP` segment が入っています。Linux x86-64 では、典型的には `/lib64/ld-linux-x86-64.so.2` がここに入ります。kernel はこれを見て、「この executable はまず dynamic loader を経由して起動しなければならない」と判断します。

つまり、動的リンク executable では kernel が直接 `main` に近いコードへ飛ぶのではありません。まず interpreter を起動経路へ載せ、その interpreter が依存関係解決と relocation を済ませてから、最終的な entry point へ制御を渡します。

初期 stack に何が積まれるか

プロセス起動時の stack には、単に return address があるわけではありません。典型的には次の情報が並びます。

- `argc`
- `argv[]`
- `envp[]`
- 補助ベクタ `auxv`

`auxv` は、page size、program header の位置、entry 情報、random bytes など、runtime が初期化に使う補助情報を渡す仕組みです。glibc や dynamic loader はこの情報を参照しながら startup を進めます。ここを知っておくと、「なぜ `main(int argc, char **argv, char **envp)` の手前にこんなに多くの層があるのか」が見えやすくなります。

`_start` は誰のコードか

実際の entry point は通常 `main` ではなく `_start` です。これは多くの場合 `crt1.o` などの startup object が提供するコードで、process 起動直後の最低限の初期化を担います。ユーザが直接書いた関数ではありませんが、C program の最初の土台として必ず通る場所です。

`_start` の役割は、すぐに何か複雑な処理をすることではありません。初期 stack から必要な情報を取り出し、glibc startup へ橋渡しし、最終的に `main` を呼べる形へ持っていくことです。つまり `_start` は「language runtime へ渡すための薄い入口」です。

`__libc_start_main` の役割

glibc を使う典型的な C program では、`_start` は最終的に `__libc_start_main` を呼びます。この関数が、libc 側の初期化、constructor 実行、`main` 呼び出し、`main` の戻り値を受けたあとの `exit` 経路をまとめます。

この 1 段があることで、ユーザは単に `main` を書くだけで済みます。しかし裏側では、runtime 初期化、環境変数の準備、destructor 処理の登録など、言語とライブラリにとって必要な前準備が済まされています。

`main` から先も startup の一部である

初学者は `main` が始まった時点で起動処理が終わったように感じがちですが、実際には `main` からの return も startup の一部です。戻り値は `exit status` として処理され、登録済み destructor や `atexit` handler の実行を経て、最終的にプロセス終了へ至ります。

この見方を持つと、program startup は「先頭に少しだけ付く序章」ではなく、`execve` から終了までを貫く runtime の導入部だと分かります。

kernel loader と dynamic loader の境界

ここで境界を明確にしておくと、責務は次のように分かれます。

- kernel: ELF executable を開き、loadable segment と初期 stack を用意し、必要なら interpreter を起動経路へ入れる
- dynamic loader: shared object を読み、動的 relocation を適用し、依存関係を解決し、entry point へ渡す
- libc startup: `main` を呼べる状態を作り、終了経路まで面倒を見る

この分業を持っているだけで、`execve`、`ld.so`、`crt1.o`、glibc startup を一つの巨大な黒箱として恐れずに済みます。

> **注意**
> 本章で扱うのは Linux x86-64 の典型経路です。`static-pie`、完全静的リンク、独自 runtime、`musl` ベースの起動経路では見え方が少し変わります。ただし、program header を見て写像し、必要なら依存解決を行い、startup object を通って言語 runtime へ渡すという骨格は共通しています。

この章の出口

本章の出口は、次の流れを一息で説明できることです。

1. `execve` が ELF を読む
2. program header に従って写像が準備される
3. 動的リンク executable なら interpreter が起動経路に入る
4. dynamic loader が依存解決と relocation を行う
5. `_start` が `__libc_start_main` へ橋渡しする
6. 最終的に `main` が呼ばれる

ここまでつながると、コンパイラ、linker、loader、runtime の境界がかなり整理されます。

付録と次の一歩

本編では、`.o` の観察から始めて、静的リンク、動的リンク、`execve`、`_start`、`main` までを一本につなぎました。この章では、復習、観察コマンドの使い分け、演習課題、用語整理、次に読む source をまとめます。リンクとローダの学習は広がりやすいですが、まずは観察の軸を固定するのが大切です。

章ごとの復習ポイント

本編の核を短く戻すと、各章は次のように整理できます。

- 導入: `compilerbook` の先にある空白は、`.o` からプロセス起動までの層である
- ELF と `.o`: section、symbol、relocation があるから `.o` は「未完成な部品」として扱える
- ABI と静的リンク: `ABI` が部品同士の約束を決め、linker が symbol 解決と relocation 適用で実行形式を閉じる
- 動的リンク: shared object、`PIC`、`GOT`、`PLT`、dynamic loader によって一部の解決を実行時へ遅らせる
- 起動経路: `execve`、program header、`PT_INTERP`、`_start`、`__libc_start_main`、`main` が連続している

この順序を頭に置いておくと、細部を忘れても全体地図を失いにくくなります。

観察コマンドの使い分け

最初に使い分けたい観察コマンドは次です。

- `readelf -h` ELF header を見る
- `readelf -S` section header を見る
- `readelf -l` program header を見る
- `readelf -s` symbol table を見る
- `readelf -r` relocation entry を見る
- `objdump -dr` 逆アセンブルしつつ relocation 位置を見る
- `nm` symbol の定義/未定義を軽く眺める
- `ldd` 実行時依存 shared object を見る

観察の基本は、「どの層の情報を見たいか」を先に決めることです。symbol と relocation を見たいのに `objdump` だけを眺め続けると、未解決参照の意味を落としやすくなります。逆に命令と call site の位置関係を知りたいのに `readelf` だけでは不十分です。

小さな演習課題

本編の理解を固めるなら、次の順で短い演習を試すとよいです。

1. 単一ファイルの `main.o` を作り、`.text`, `.data`, `.bss`, `.rodata` がどれだけ現れるか観察する
2. 複数ファイルへ分け、未定義 symbol と relocation entry を確認する
3. `ar` で静的ライブラリを作り、link 順序の違いで結果が変わる例を試す
4. `-fPIC` 付き shared object を作り、`readelf -r` と `objdump -d` で `GOT/PLT` を観察する
5. `readelf -l` と `readelf -S` を見比べ、section と segment の違いを言葉で説明する
6. `_start`、`__libc_start_main`、`main` のつながりを `objdump` と symbol 表でたどる

どの課題も、巨大なアプリケーションである必要はありません。10 行前後の小さな C program で十分です。大事なのは、出力を見て「今どの層を見ているか」を言い分けることです。

この章の到達点

本書の理解確認として、最低限次を説明できるかを試してください。

- なぜ `.o` には relocation が必要なのか
- `PLT` と `GOT` はそれぞれ何のためにあるのか
- なぜ `main` ではなく `_start` が entry point なのか
- section header と program header は誰のための情報なのか

よくあるつまずき

初学者が特につまずきやすい点を挙げます。

`.text` を読めば実行時の全体が分かると思ってしまう

分かるのは命令列の一部だけです。実行時の写像は program header、動的依存は dynamic section、起動経路は startup object と loader まで見ないとつながりません。

`GOT` と symbol table を混同する

symbol table は名前と属性の表です。`GOT` は実行時に使うアドレス表です。役割が違います。

PLT を単なる無駄なジャンプだと思ってしまう

PLT は lazy binding を支える入口です。初回呼び出しと 2 回目以降で意味が変わる点が重要です。

section と segment を混同する

section は static link の都合、segment は load の都合です。対象読者の頭の中でも、この二層を分けておく必要があります。

main が最初の関数だと思ってしまう

main の前には _start と runtime 初期化があります。main の後にも exit 経路があります。

用語小事典

- ELF 実行形式、shared object、.o などに使われる形式
- section linker が部品を扱うための単位
- segment loader が写像を扱うための単位
- symbol 名前と属性を持つ参照/定義の単位
- relocation 後で値を書き換えるための指示
- ABI バイナリ互換のための約束
- PIC ロード位置に依存しにくいコード生成方針
- GOT 実行時に確定するアドレスを置く表
- PLT shared object 越しの関数呼び出しを仲介する stub 群
- dynamic loader shared object のロードと動的 relocation を担う実行時ローダ
- _start 実行開始直後の入口
- crt1.o _start を提供する startup object の代表
- __libc_start_main glibc 側の startup 本体
- auxv kernel から runtime へ渡される補助情報

次に読むなら

本書の次に進むなら、方向は大きく 3 つあります。

第一は linker 自体を読む方向です。GNU ld や lld の source、linker script、relocation 実装を追うと、静的リンク理解が一段深まります。

第二は libc と startup を読む方向です。glibc の csu、ld.so、musl の startup を比較すると、_start から main への橋渡しがより具体的になります。

第三は OS 側ローダを読む方向です。Linux kernel の execve 経路や binfmt_elf を読むと、「ユーザ空間へ制御を渡す直前まで」に関心を広げられます。

参考資料

本書を進める際の主な資料は次です。

- ELF gABI
- System V AMD64 psABI
- man 5 elf
- man 2 execve
- ld.so(8)
- GNU binutils documentation
- glibc startup / dynamic loader source
- lld design and source comments

編集注記
本書は Linux x86-64 と GNU 系 toolchain に話を固定しました。これは一般性を捨てたのではなく、最初の 1 冊で「何が主役か」を見失わないためです。別環境へ広げるのは、この地図を持ったあとで十分です。

おわりに

リンクとローダの層が見えるようになると、コンパイラ本で学んだ symbol や呼び出し規約が、急に現実の executable と結び付きます。逆に runtime や OS の本を読むときも、`PLT/GOT` や `_start` の意味が見えているだけで怖さがかなり減ります。

本書が目指したのは、仕様の全網羅ではなく、`.o` から `main` までの一本の線を持てるようにすることでした。ここまで読めたなら、linker、loader、runtime のどこへ進んでも、少なくとも地図を見失わずに歩けるはずです。